Date.prototype.getTime()

📄 Título

Date.prototype.getTime() – El timestamp universal en milisegundos

---

📝 Definición

`Date.prototype.getTime()` es un método de JavaScript que devuelve el **número de milisegundos** transcurridos desde el **1 de enero de 1970, 00:00:00 UTC** (conocido como el **Unix Epoch**) hasta la fecha representada por el objeto `Date`.

🧠 ¿Qué es?

Es el método que proporciona la representación **numérica universal** de una fecha en JavaScript. El valor devuelto es independiente de la zona horaria y representa un **instante exacto en el tiempo**, medido en milisegundos.

Características clave:

- El timestamp devuelto es **siempre en UTC** (no depende de la zona horaria local).
- Es un número entero de 64 bits (en JavaScript, Number de precisión doble).
- Los valores negativos representan fechas anteriores al 1 de enero de 1970.
- Es el formato utilizado para **comparar fechas** y realizar **cálculos temporales**.

🎯 ¿Para qué sirve?

- **Comparar fechas** de manera precisa y sencilla.
- Calcular **diferencias de tiempo** entre dos fechas.
- Medir **tiempos de ejecución** y **rendimiento**.
- Generar **identificadores únicos** basados en tiempo.

- Almacenar fechas en **bases de datos** como números (más eficiente).
- Crear **timestamps** para **logs** y **auditoría**.
- Implementar **cachés con expiración** basada en tiempo.
- Sincronizar **eventos y temporizadores**.
- Convertir entre diferentes **zonas horarias**.

⚙️ ¿Cómo funciona?

El método devuelve el **timestamp** que representa el número de milisegundos desde el Epoch.

Detalles técnicos:

- El Epoch es el 1 de enero de 1970, 00:00:00 UTC.
- Un día tiene 86,400,000 milisegundos (24 * 60 * 60 * 1000).
- El rango de fechas soportado es aproximadamente desde el año -271,821 hasta el 275,760.
- El timestamp es un **valor primitivo** (número), no un objeto.

Equivalencias:

- `date.getTime()` ≈ `date.valueOf()`
- `date.getTime()` ≈ `+date` (conversión a número)
- `date.getTime()` ≈ `Date.parse(date.toString())`

📝 Sintaxis / Estructura (JavaScript)

```
date.getTime()
```

Retorna:

- Un número que representa los milisegundos desde el Epoch.

Errores:

- Si el objeto `Date` es inválido, devuelve `NaN`.

💡 Ejemplos

```javascript
// 1. Uso básico
const date = new Date(2024, 11, 25);
console.log(date.getTime());
// 1735084800000 (timestamp para 2024-12-25 00:00:00 UTC)

// 2. Fecha actual
const now = new Date();
console.log(now.getTime()); // Timestamp actual en milisegu
ndos

// 3. Comparación de fechas
const date1 = new Date(2024, 0, 1);
const date2 = new Date(2024, 11, 31);

if (date1.getTime() < date2.getTime()) {
  console.log('date1 es anterior a date2');
}

// 4. Calcular diferencia en días
function daysBetween(date1, date2) {
  const diff = Math.abs(date2.getTime() - date1.getTime());
  return Math.floor(diff / (1000 * 60 * 60 * 24));
}

console.log(daysBetween(
  new Date(2024, 0, 1),
  new Date(2024, 11, 31)
)); // 365

// 5. Tiempo de ejecución
const start = new Date().getTime();

// Simular operación costosa
for (let i = 0; i < 1000000; i++) {
  Math.sqrt(i);
}
```

```
const end = new Date().getTime();
console.log(`Tiempo de ejecución:${end - start}ms`);

// 6. Performance.now() vs getTime()
console.log('getTime():', new Date().getTime());
console.log('performance.now():', performance.now());

// 7. Timestamp para identificadores únicos
function generateId() {
  return `${Date.now()}-${Math.random().toString(36).substr
(2, 6)}`;
}

console.log(generateId()); // "1703539200000-abc123"

// 8. Fechas antes del Epoch
const oldDate = new Date(1969, 6, 20);
console.log(oldDate.getTime()); // Número negativo: -152064
00000

// 9. Timestamp a Date
const timestamp = 1735084800000;
const restoredDate = new Date(timestamp);
console.log(restoredDate.toISOString()); // "2024-12-25T00:
00:00.000Z"

// 10. Equivalencia de métodos
const d = new Date();
console.log(d.getTime() === d.valueOf()); // true
console.log(d.getTime() === +d); // true

// 11. Ordenar fechas por timestamp
const dates = [
  new Date(2024, 5, 15),
  new Date(2023, 11, 25),
  new Date(2024, 0, 1)
];
```

```
dates.sort((a, b) => a.getTime() - b.getTime());
console.log(dates.map(d => d.toISOString()));

// 12. Cache con expiración
class TimedCache {
  constructor(ttl = 60000) { // 1 minuto por defecto
    this.cache = new Map();
    this.ttl = ttl;
  }

  set(key, value) {
    this.cache.set(key, {
      value,
      timestamp: Date.now()
    });
  }

  get(key) {
    const entry = this.cache.get(key);
    if (!entry) return null;

    if (Date.now() - entry.timestamp > this.ttl) {
      this.cache.delete(key);
      return null;
    }

    return entry.value;
  }
}

const cache = new TimedCache(5000);
cache.set('test', 'valor');
console.log(cache.get('test')); // "valor"

// 13. Días desde el Epoch
function daysSinceEpoch(date = new Date()) {
  return Math.floor(date.getTime() / (1000 * 60 * 60 * 2
```

```
4));
}

console.log(daysSinceEpoch()); // Días desde 1970-01-01

// 14. Horas desde el Epoch
function hoursSinceEpoch(date = new Date()) {
  return Math.floor(date.getTime() / (1000 * 60 * 60));
}

console.log(hoursSinceEpoch());

// 15. Timestamp legible (formato Unix clásico)
function getUnixTimestamp(date = new Date()) {
  return Math.floor(date.getTime() / 1000);
}

console.log(getUnixTimestamp()); // Segundos desde el Epoch

// 16. Validar si un timestamp es válido
function isValidTimestamp(timestamp) {
  const date = new Date(timestamp);
  return date.getTime() === timestamp && !isNaN(timestamp);
}

console.log(isValidTimestamp(1735084800000)); // true
console.log(isValidTimestamp(-1)); // true (fecha antes del
Epoch)
console.log(isValidTimestamp(NaN)); // false

// 17. Conversión a diferentes unidades
function convertTimestamp(timestamp) {
  return {
    milliseconds: timestamp,
    seconds: Math.floor(timestamp / 1000),
    minutes: Math.floor(timestamp / (1000 * 60)),
    hours: Math.floor(timestamp / (1000 * 60 * 60)),
    days: Math.floor(timestamp / (1000 * 60 * 60 * 24)),
```

```
    weeks: Math.floor(timestamp / (1000 * 60 * 60 * 24 *
7)),
    years: Math.floor(timestamp / (1000 * 60 * 60 * 24 * 36
5.25))
  };
}

console.log(convertTimestamp(Date.now()));

// 18. Timestamp para nombre de archivo
function getTimestampFileName() {
  const now = new Date();
  const year = now.getFullYear();
  const month = String(now.getMonth() + 1).padStart(2,
'0');
  const day = String(now.getDate()).padStart(2, '0');
  const hours = String(now.getHours()).padStart(2, '0');
  const minutes = String(now.getMinutes()).padStart(2,
'0');
  const seconds = String(now.getSeconds()).padStart(2,
'0');

  return `${year}${month}${day}_${hours}${minutes}${second
s}`;
}

console.log(getTimestampFileName()); // "20241225_153045"

// 19. Middleware para logging
function logRequest(req, res, next) {
  const start = Date.now();

  res.on('finish', () => {
    const duration = Date.now() - start;
    console.log(`[${new Date().toISOString()}]${req.method}
${req.url} -${duration}ms`);
  });
```

```
  next();
}

// 20. Generar rango de timestamps
function generateTimestampRange(startDate, endDate, interva
lMinutes = 60) {
  const start = new Date(startDate).getTime();
  const end = new Date(endDate).getTime();
  const interval = intervalMinutes * 60 * 1000;

  const timestamps = [];
  let current = start;

  while (current <= end) {
    timestamps.push(current);
    current += interval;
  }

  return timestamps;
}

console.log(generateTimestampRange('2024-12-25', '2024-12-2
6', 360));
// Timestamps cada 6 horas
```

⚠️ Errores comunes

- **No considerar la zona horaria**: `getTime()` devuelve UTC, por lo que no hay problema.
- **Crear nuevos objetos Date innecesariamente**:

```
// ❌ MAL
function getTime() {
  return new Date().getTime();
}
```

```
// ✅ BIEN
function getTime() {
  return Date.now();
}
```

- **Comparar fechas con `==` o `===`:**

```
// ❌ MAL
const d1 = new Date(2024, 0, 1);
const d2 = new Date(2024, 0, 1);
console.log(d1 === d2); // false (objetos diferentes)

// ✅ BIEN
console.log(d1.getTime() === d2.getTime()); // true
```

- **No usar `Date.now()` para el timestamp actual:**

```
// ❌ MENOS EFICIENTE
const now = new Date().getTime();

// ✅ MÁS EFICIENTE
const now = Date.now();
```

- **Pensar que `getTime()` devuelve segundos**: Devuelve **milisegundos**, no segundos.

💼 Casos de uso

1. **Sistema de rate limiting basado en tiempo:**

```
class RateLimiter {
  constructor(limit = 100, windowMs = 60000) {
    this.limit = limit;
    this.windowMs = windowMs;
```

```
    this.requests = [];
  }

  isAllowed() {
    const now = Date.now();

    // Eliminar solicitudes antiguas
    this.requests = this.requests.filter(
      timestamp => now - timestamp < this.windowMs
    );

    if (this.requests.length >= this.limit) {
      return false;
    }

    this.requests.push(now);
    return true;
  }

  getRemaining() {
    const now = Date.now();
    this.requests = this.requests.filter(
      timestamp => now - timestamp < this.windowMs
    );
    return this.limit - this.requests.length;
  }

  getResetTime() {
    if (this.requests.length === 0) return 0;
    const oldest = this.requests[0];
    const now = Date.now();
    return Math.max(0, this.windowMs - (now - oldest));
  }
}

const limiter = new RateLimiter(3, 10000); // 3 solicitu
des en 10 segundos
console.log(limiter.isAllowed()); // true
```

```
console.log(limiter.isAllowed()); // true
console.log(limiter.isAllowed()); // true
console.log(limiter.isAllowed()); // false
console.log(limiter.getRemaining()); // 0
console.log(limiter.getResetTime()); // tiempo restante
```

2. **Sistema de caché con expiración por timestamp**:

```
class CacheManager {
  constructor(defaultTTL = 3600000) { // 1 hora
    this.cache = new Map();
    this.defaultTTL = defaultTTL;
  }

  set(key, value, ttl = this.defaultTTL) {
    this.cache.set(key, {
      value,
      expires: Date.now() + ttl
    });
  }

  get(key) {
    const entry = this.cache.get(key);
    if (!entry) return null;

    if (Date.now() > entry.expires) {
      this.cache.delete(key);
      return null;
    }

    return entry.value;
  }

  clear() {
    this.cache.clear();
  }
```

```
  getStats() {
    const now = Date.now();
    let total = 0;
    let expired = 0;

    for (const [key, entry] of this.cache) {
      total++;
      if (now > entry.expires) {
        expired++;
      }
    }

    return { total, expired, active: total - expired };
  }

  cleanup() {
    const now = Date.now();
    for (const [key, entry] of this.cache) {
      if (now > entry.expires) {
        this.cache.delete(key);
      }
    }
  }
}

const cache = new CacheManager(5000);
cache.set('key1', 'valor1');
cache.set('key2', 'valor2', 3000);
console.log(cache.get('key1')); // "valor1"

setTimeout(() => {
  console.log(cache.get('key2')); // null (expirado)
  console.log(cache.getStats());
}, 4000);
```

3. **Sistema de tokens con expiración**:

```
class TokenManager {
  constructor(expirationMs = 3600000) { // 1 hora
    this.tokens = new Map();
    this.expirationMs = expirationMs;
  }

  generateToken(userId) {
    const token = this.createToken();
    const expires = Date.now() + this.expirationMs;

    this.tokens.set(token, {
      userId,
      createdAt: Date.now(),
      expires
    });

    return { token, expires };
  }

  createToken() {
    return Buffer.from(
      `${Date.now()}-${Math.random().toString(36)}`
    ).toString('base64');
  }

  validateToken(token) {
    const entry = this.tokens.get(token);
    if (!entry) return null;

    if (Date.now() > entry.expires) {
      this.tokens.delete(token);
      return null;
    }

    return entry.userId;
  }

  refreshToken(token) {
```

```
    const entry = this.tokens.get(token);
    if (!entry) return null;

    const newExpires = Date.now() + this.expirationMs;
    this.tokens.set(token, { ...entry, expires: newExpir
es });
    return newExpires;
  }

  revokeToken(token) {
    this.tokens.delete(token);
  }

  cleanup() {
    const now = Date.now();
    for (const [token, entry] of this.tokens) {
      if (now > entry.expires) {
        this.tokens.delete(token);
      }
    }
  }
}

const tokenManager = new TokenManager(10000);
const { token, expires } = tokenManager.generateToken('u
ser123');
console.log(tokenManager.validateToken(token)); // "user
123"

setTimeout(() => {
  console.log(tokenManager.validateToken(token)); // nul
l (expirado)
}, 11000);
```

4. **Sistema de pruebas A/B con temporización**:

```
class ABTesting {
  constructor() {
    this.experiments = new Map();
  }

  startExperiment(name, variantA, variantB, durationMs =
86400000) {
    const start = Date.now();
    this.experiments.set(name, {
      start,
      end: start + durationMs,
      variants: [variantA, variantB],
      results: { A: 0, B: 0 }
    });
  }

  getVariant(name) {
    const experiment = this.experiments.get(name);
    if (!experiment) return null;

    const now = Date.now();
    if (now > experiment.end) {
      return null; // Experimento terminado
    }

    // Asignación determinística basada en tiempo
    const elapsed = now - experiment.start;
    const variant = Math.floor(elapsed / 1000) % 2 === 0
? 'A' : 'B';
    return variant;
  }

  recordResult(name, variant) {
    const experiment = this.experiments.get(name);
    if (!experiment) return;

    const now = Date.now();
    if (now > experiment.end) return;
```

```
    experiment.results[variant] += 1;
  }

  getResults(name) {
    const experiment = this.experiments.get(name);
    if (!experiment) return null;

    const now = Date.now();
    const elapsed = now - experiment.start;
    const remaining = experiment.end - now;

    return {
      name,
      start: experiment.start,
      end: experiment.end,
      remaining: Math.max(0, remaining),
      elapsed,
      progress: (elapsed / (experiment.end - experiment.
start)) * 100,
      results: experiment.results
    };
  }
}

const ab = new ABTesting();
ab.startExperiment('test1', 'control', 'variante', 3000
0);
console.log(ab.getVariant('test1')); // "A" o "B"
```

5. **Medición de tiempos de respuesta en API**:

```
class PerformanceMonitor {
  constructor() {
    this.metrics = [];
    this.maxMetrics = 1000;
  }
```

```
  measureEndpoint(name, fn) {
    return async (...args) => {
      const start = performance.now();
      try {
        const result = await fn(...args);
        const duration = performance.now() - start;
        this.recordMetric(name, duration, true);
        return result;
      } catch (error) {
        const duration = performance.now() - start;
        this.recordMetric(name, duration, false);
        throw error;
      }
    };
  }

  recordMetric(name, duration, success) {
    this.metrics.push({
      name,
      duration,
      success,
      timestamp: Date.now()
    });

    if (this.metrics.length > this.maxMetrics) {
      this.metrics = this.metrics.slice(-this.maxMetric
s);
    }
  }

  getStats(name, timeWindow = 60000) {
    const now = Date.now();
    const windowMetrics = this.metrics.filter(m =>
      m.name === name && now - m.timestamp < timeWindow
    );

    if (windowMetrics.length === 0) return null;
```

```
    const durations = windowMetrics.map(m => m.duratio
n);
    const successRate = windowMetrics.filter(m => m.succ
ess).length / windowMetrics.length;

    return {
      name,
      count: windowMetrics.length,
      avgDuration: durations.reduce((a, b) => a + b, 0)
/ durations.length,
      minDuration: Math.min(...durations),
      maxDuration: Math.max(...durations),
      p95: this.calculatePercentile(durations, 95),
      successRate: successRate * 100,
      errors: windowMetrics.filter(m => !m.success).leng
th
    };
  }

  calculatePercentile(sortedValues, percentile) {
    if (sortedValues.length === 0) return 0;
    const sorted = [...sortedValues].sort((a, b) => a -
b);
    const index = Math.ceil((percentile / 100) * sorted.
length) - 1;
    return sorted[index];
  }

  clearMetrics() {
    this.metrics = [];
  }
}

// Simular uso
const monitor = new PerformanceMonitor();
const wrapped = monitor.measureEndpoint('api.test', asyn
c () => {
```

```
  await new Promise(resolve => setTimeout(resolve, 10
0));
  return 'resultado';
});
```

6. **Sistema de notificaciones programadas**:

```
class ScheduledNotifications {
  constructor() {
    this.notifications = [];
    this.interval = null;
  }

  schedule(title, body, scheduledTime) {
    const timestamp = typeof scheduledTime === 'number'
      ? scheduledTime
      : new Date(scheduledTime).getTime();

    const notification = {
      id: Date.now() + '-' + Math.random().toString(36).
substr(2, 6),
      title,
      body,
      scheduled: timestamp,
      triggered: false
    };

    this.notifications.push(notification);
    this.notifications.sort((a, b) => a.scheduled - b.sc
heduled);
    return notification;
  }

  start() {
    this.interval = setInterval(() => {
      const now = Date.now();
      const toTrigger = this.notifications.filter(
```

```
        n => !n.triggered && n.scheduled <= now
      );

      toTrigger.forEach(n => {
        this.triggerNotification(n);
        n.triggered = true;
      });

      // Limpiar notificaciones antiguas
      this.notifications = this.notifications.filter(
        n => !n.triggered || n.scheduled > now - 8640000
0
      );
    }, 1000);
  }

  triggerNotification(notification) {
    console.log(`🔔 NOTIFICACIÓN:${notification.title}\n
${notification.body}`);
    // En una app real, mostrar notificación
  }

  getPending() {
    const now = Date.now();
    return this.notifications
      .filter(n => !n.triggered)
      .map(n => ({
        ...n,
        remainingMs: n.scheduled - now,
        remainingText: this.formatTime(n.scheduled - no
w)
      }));
  }

  formatTime(ms) {
    if (ms <= 0) return 'Ahora';
    const seconds = Math.floor(ms / 1000);
    if (seconds < 60) return `${seconds}s`;
```

```
    const minutes = Math.floor(seconds / 60);
    if (minutes < 60) return `${minutes}m`;
    const hours = Math.floor(minutes / 60);
    if (hours < 24) return `${hours}h`;
    const days = Math.floor(hours / 24);
    return `${days}d`;
  }

  stop() {
    if (this.interval) {
      clearInterval(this.interval);
      this.interval = null;
    }
  }
}

const notifications = new ScheduledNotifications();
notifications.schedule('Reunión', 'Reunión en 5 minuto
s', Date.now() + 300000);
notifications.schedule('Cumpleaños', 'Feliz cumpleaños',
Date.now() + 600000);
notifications.start();
console.log(notifications.getPending());
```

7. **Sistema de auditoría con timestamps**:

```
class AuditLogger {
  constructor() {
    this.logs = [];
    this.maxLogs = 10000;
  }

  log(action, details, userId = null) {
    const entry = {
      id: this.generateId(),
      action,
      details,
```

```
      userId,
      timestamp: Date.now(),
      isoDate: new Date().toISOString(),
      timestampMs: Date.now()
    };

    this.logs.push(entry);

    if (this.logs.length > this.maxLogs) {
      this.logs = this.logs.slice(-this.maxLogs);
    }

    return entry;
  }

  generateId() {
    return `${Date.now()}-${Math.random().toString(36).s
ubstr(2, 8)}`;
  }

  getLogs(filters = {}) {
    let filtered = this.logs;

    if (filters.action) {
      filtered = filtered.filter(l => l.action === filte
rs.action);
    }

    if (filters.userId) {
      filtered = filtered.filter(l => l.userId === filte
rs.userId);
    }

    if (filters.startTime) {
      filtered = filtered.filter(l => l.timestamp >= fil
ters.startTime);
    }
```

```
    if (filters.endTime) {
      filtered = filtered.filter(l => l.timestamp <= fil
ters.endTime);
    }

    return filtered;
  }

  getTimeRange(startTime, endTime) {
    return this.getLogs({ startTime, endTime });
  }

  getStats(timeWindow = 3600000) { // 1 hora
    const now = Date.now();
    const windowLogs = this.logs.filter(l => now - l.tim
estamp < timeWindow);

    const actions = {};
    windowLogs.forEach(l => {
      actions[l.action] = (actions[l.action] || 0) + 1;
    });

    return {
      total: windowLogs.length,
      timeWindow,
      actions,
      mostActive: Object.entries(actions).sort((a, b) =>
b[1] - a[1])[0]
    };
  }

  clear() {
    this.logs = [];
  }
}

const audit = new AuditLogger();
audit.log('login', { ip: '192.168.1.1', userAgent: 'Chro
```

```
me' }, 'user123');
audit.log('page_view', { page: '/dashboard' }, 'user12
3');
audit.log('logout', { reason: 'user_request' }, 'user12
3');
console.log(audit.getStats());
```

8. **Sistema de versionado de archivos**:

```
class FileVersioning {
  constructor() {
    this.files = new Map();
  }

  saveFile(name, content, metadata = {}) {
    const timestamp = Date.now();
    const version = `${name}_${timestamp}`;

    if (!this.files.has(name)) {
      this.files.set(name, { versions: [], current: null
});
    }

    const file = this.files.get(name);
    const versionEntry = {
      version,
      content,
      timestamp,
      metadata: {
        ...metadata,
        savedAt: new Date(timestamp).toISOString()
      }
    };

    file.versions.push(versionEntry);
    file.current = versionEntry;
```

```
    return versionEntry;
  }

  getVersion(name, timestamp) {
    const file = this.files.get(name);
    if (!file) return null;

    return file.versions.find(v => v.timestamp === times
tamp) || null;
  }

  getVersions(name) {
    const file = this.files.get(name);
    if (!file) return [];
    return file.versions.map(v => ({
      version: v.version,
      timestamp: v.timestamp,
      metadata: v.metadata
    }));
  }

  getCurrentVersion(name) {
    const file = this.files.get(name);
    if (!file) return null;
    return file.current;
  }

  diffVersions(name, ts1, ts2) {
    const v1 = this.getVersion(name, ts1);
    const v2 = this.getVersion(name, ts2);
    if (!v1 || !v2) return null;

    // Simular diff simple
    const lines1 = v1.content.split('\n');
    const lines2 = v2.content.split('\n');
    const diff = lines2.filter(line => !lines1.includes
(line));
```

```
    return {
      version1: ts1,
      version2: ts2,
      addedLines: diff,
      totalAdded: diff.length
    };
  }

  rollbackTo(name, timestamp) {
    const file = this.files.get(name);
    if (!file) return null;

    const version = file.versions.find(v => v.timestamp
=== timestamp);
    if (!version) return null;

    file.current = version;
    return version;
  }
}

const vcs = new FileVersioning();
vcs.saveFile('doc.txt', 'Versión 1', { author: 'Juan'
});
setTimeout(() => {
  vcs.saveFile('doc.txt', 'Versión 2', { author: 'Juan'
});
}, 1000);
setTimeout(() => {
  vcs.saveFile('doc.txt', 'Versión 3', { author: 'María'
});
}, 2000);
```

🧩 Relación con otros conceptos

- `Date.now()`: Método estático que devuelve el timestamp actual.
- `Date.parse()`: Convierte string de fecha a timestamp.

- `Date.valueOf()`: Similar a `getTime()`.
- `Date.UTC()`: Obtiene timestamp para fecha UTC específica.
- `performance.now()`: Timestamp de alta resolución para mediciones.
- `process.hrtime()`: Para mediciones de alta precisión en Node.js.

🧠 Analogía

Imagina que `getTime()` es como un **cuentakilómetros del tiempo**. Mide cuántos milisegundos han pasado desde un punto de referencia fijo (el 1 de enero de 1970). Es como un cronómetro universal que todos los sistemas pueden usar para ponerse de acuerdo sobre qué hora es.

📚 Fuente

- MDN Web Docs – Date.prototype.getTime()
- ECMAScript Specification – Date.prototype.getTime
- Unix Time
- MDN – Date.now()